

Key Drivers Analysis

Zhutong Zhang

2025-06-05

1. Introduction

In this project, we explore two fundamental branches of machine learning—unsupervised and supervised learning—by implementing multiple algorithms from scratch. Our goal is to understand not only how these methods work, but also when and why they are effective tools for marketing analytics.

We begin with **K-Means Clustering**, an **unsupervised learning** algorithm that helps uncover hidden segments in data. Using the well-known **penguins** dataset, we identify natural groupings based on physical measurements such as bill length and flipper size. This mirrors real-world business applications like customer segmentation.

Next, we examine **two complementary supervised learning methods**:

- **Key Drivers Analysis**: We apply a range of regression-based techniques—including Pearson R^2 , standardized coefficients, usefulness (ΔR^2), SHAP values, and tree-based variable importance—to determine which customer perceptions most influence satisfaction. This analysis reflects real-world business needs to **explain** and **prioritize** actionable factors.
- **K-Nearest Neighbors (KNN)**: We also implement KNN for **classification** on a yogurt purchase dataset. By visualizing how the model behaves under different values of K (e.g., $K=1$ and $K=15$), we explore model flexibility and the bias-variance tradeoff, as illustrated in Slide 19 from class.

By combining clustering, interpretation, and classification techniques, we reinforce our understanding of distance-based learning methods, explore their assumptions, and assess their performance across a variety of practical contexts.

Part 1: K-Means Clustering (Unsupervised Learning)

Dataset Overview

```
import pandas as pd

penguins_df = pd.read_csv("palmer_penguins.csv")

penguins_df.head()
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181	3750	male
1	Adelie	Torgersen	39.5	17.4	186	3800	female
2	Adelie	Torgersen	40.3	18.0	195	3250	female
3	Adelie	Torgersen	36.7	19.3	193	3450	female
4	Adelie	Torgersen	39.3	20.6	190	3650	male

We begin our analysis using the `palmer_penguins` dataset, a popular alternative to the Iris dataset in classification and clustering problems. It contains biological measurements of three penguin species—**Adelie**, **Chinstrap**, and **Gentoo**—collected from the Palmer Archipelago in Antarctica.

Key numerical features include: - **Bill Length (mm)** - **Bill Depth (mm)** - **Flipper Length (mm)** - **Body Mass (g)**

In this unsupervised learning task, we ignore the species labels and attempt to identify natural clusters using only these four physical features. This mirrors a typical business scenario where we aim to discover customer segments without prior knowledge of group membership.

K-Means Clustering: Algorithm by Hand

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
import seaborn as sns
```

```

# Select features for K-Means clustering
features = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
penguins_features = penguins_df[features].dropna()

# Standardize the features
scaler = StandardScaler()
X = scaler.fit_transform(penguins_features)

# Implement K-Means manually
def initialize_centroids(X, k):
    np.random.seed(42)
    random_indices = np.random.permutation(X.shape[0])[:k]
    return X[random_indices]

def assign_clusters(X, centroids):
    distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
    return np.argmin(distances, axis=1)

def update_centroids(X, labels, k):
    return np.array([X[labels == i].mean(axis=0) for i in range(k)])

# Run the algorithm
def kmeans(X, k=3, max_iters=100, tol=1e-4):
    centroids = initialize_centroids(X, k)
    for i in range(max_iters):
        labels = assign_clusters(X, centroids)
        new_centroids = update_centroids(X, labels, k)
        if np.allclose(centroids, new_centroids, atol=tol):
            break
        centroids = new_centroids
    return labels, centroids

# Apply K-Means
labels, final_centroids = kmeans(X, k=3)

# Add cluster labels to original DataFrame
penguins_clustered = penguins_features.copy()
penguins_clustered['cluster'] = labels

import seaborn as sns
import pandas as pd

```

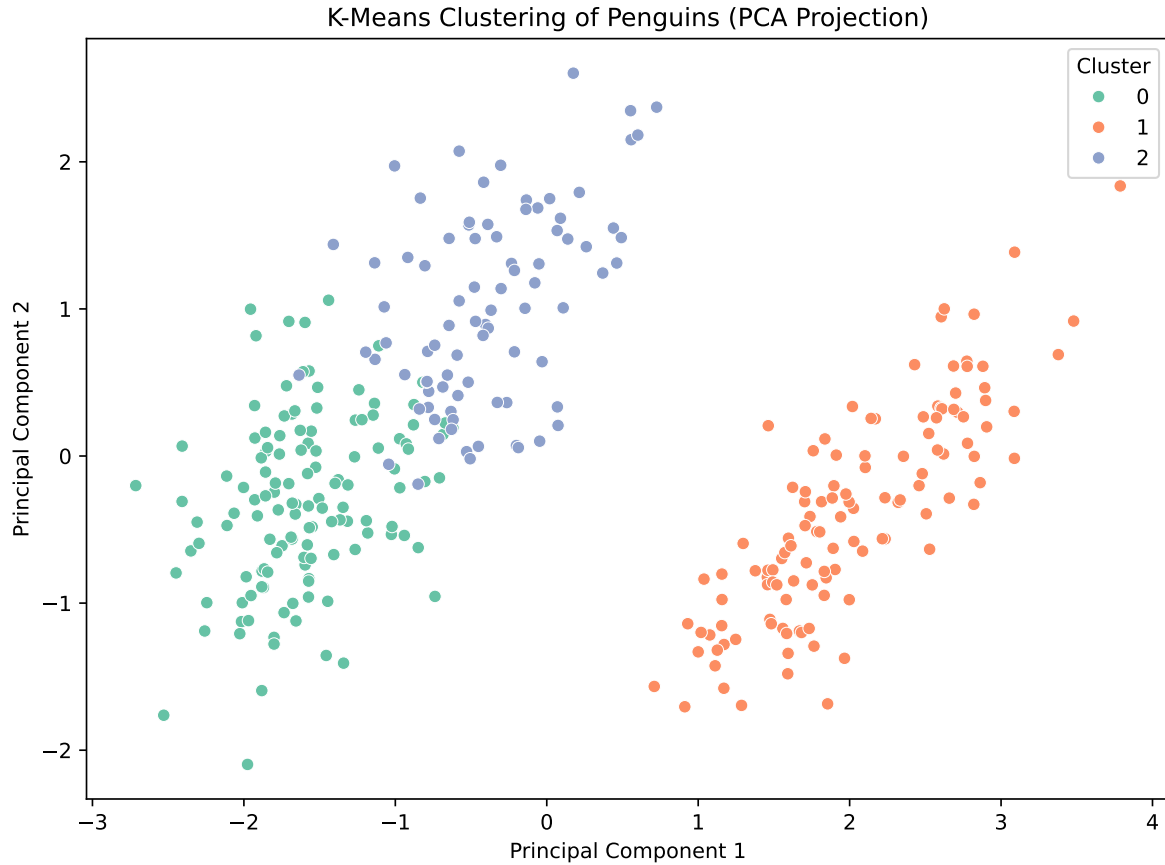
```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA

# Reduce to 2D using PCA for visualization
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X)
penguins_clustered['PC1'] = X_2d[:, 0]
penguins_clustered['PC2'] = X_2d[:, 1]

# Plot clustering result
plt.figure(figsize=(8, 6))
sns.scatterplot(data=penguins_clustered, x='PC1', y='PC2', hue='cluster', palette='Set2')
plt.title('K-Means Clustering of Penguins (PCA Projection)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title='Cluster')
plt.tight_layout()
plt.show()

```



K-Means clustering partitions observations into (K) clusters such that each observation belongs to the cluster with the nearest mean. Here's how the algorithm works:

1. **Initialization:** Randomly assign (K) initial cluster centers (centroids).
2. **Assignment Step:** For each data point, compute the distance to each centroid and assign the point to the nearest one.
3. **Update Step:** Recalculate each centroid as the mean of all points assigned to its cluster.
4. **Repeat:** Iterate the assignment and update steps until the cluster assignments no longer change or the maximum number of iterations is reached.

We implement this algorithm from scratch, using only basic Python operations, without relying on `sklearn.cluster.KMeans`. This allows us to observe the inner workings of the algorithm—especially how centroids shift and clusters stabilize over time.

Part 2: Key Drivers Analysis (Supervised Learning)

Dataset Overview

```
import pandas as pd

yogurt_df = pd.read_csv("yogurt_data.csv")
drivers_df = pd.read_csv("data_for_drivers_analysis.csv")

drivers_df.head()
```

	brand	id	satisfaction	trust	build	differs	easy	appealing	rewarding	popular	service	impa
0	1	98	3	1	0	1	1	1	0	0	1	0
1	1	179	5	0	0	0	0	0	0	0	0	0
2	1	197	3	1	0	0	1	1	1	0	1	1
3	1	317	1	0	0	0	0	1	0	1	1	1
4	1	356	4	1	1	1	1	1	1	1	1	1

```
yogurt_df.head()
```

	id	y1	y2	y3	y4	f1	f2	f3	f4	p1	p2	p3	p4
0	1	0	0	0	1	0	0	0	0	0.108	0.081	0.061	0.079
1	2	0	1	0	0	0	0	0	0	0.108	0.098	0.064	0.075
2	3	0	1	0	0	0	0	0	0	0.108	0.098	0.061	0.086
3	4	0	1	0	0	0	0	0	0	0.108	0.098	0.061	0.086
4	5	0	1	0	0	0	0	0	0	0.125	0.098	0.049	0.079

For this analysis, we use a customer satisfaction dataset where the target variable is a numeric score (**satisfaction**) ranging from 1 to 5. The input features represent customer perceptions, such as whether the product is trustworthy, appealing, easy to use, etc. All features are binary (0 or 1), indicating agreement with each perception item.

Variable Importance Methods

We compare six variable importance techniques to determine which features most influence satisfaction:

- **Pearson R^2 :** Measures how strongly each variable is correlated with the response.
 - **Standardized Coefficients:** From a linear regression with standardized inputs.
 - **Usefulness (ΔR^2):** The drop in R^2 from removing a variable from the model.
 - **SHAP (Linear):** Shapley values approximated for a linear model.
 - **Johnson's Relative Weights:** A fast, algebraic approximation of Shapley values for regression.
 - **Random Forest Gini Importance:** Based on how much each feature reduces impurity across decision trees.
-

Key Drivers Table

The table below shows the percentage importance of each variable under the six methods. The values are normalized so that each column sums to 100%.

```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.base import clone
import shap

# Prepare data for supervised learning
drivers_clean = drivers_df.dropna()
X = drivers_clean.drop(columns=['brand', 'id', 'satisfaction'])
y = drivers_clean['satisfaction']

# Standardize features for linear methods
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_df = pd.DataFrame(X_scaled, columns=X.columns)

# 1. Pearson Correlations ( $R^2$ )
pearson_r2 = X_df.corrwith(y).pow(2)
```

```

# 2. Standardized Coefficients
reg = LinearRegression().fit(X_scaled, y)
std_coefs = pd.Series(reg.coef_ * X_df.std(), index=X.columns)

# 3. Usefulness:  $\Delta R^2$  from dropping each variable
full_r2 = r2_score(y, reg.predict(X_scaled))
usefulness = {}
for col in X.columns:
    cols_minus = [c for c in X.columns if c != col]
    X_minus = X_df[cols_minus]
    r2_minus = r2_score(y, LinearRegression().fit(X_minus, y).predict(X_minus))
    usefulness[col] = full_r2 - r2_minus
usefulness = pd.Series(usefulness)

# 4. SHAP Values for Linear Regression
explainer = shap.Explainer(reg, X_df)
shap_values = explainer(X_df)
shap_importance = pd.DataFrame(shap_values.values, columns=X.columns).abs().mean()

# 5. Johnson's Epsilon (approx. via squared beta * variance of X)
johnson = pd.Series(reg.coef_**2 * X_df.var().values, index=X.columns)

# 6. Random Forest Gini Importance
rf = RandomForestRegressor(random_state=42).fit(X, y)
rf_importance = pd.Series(rf.feature_importances_, index=X.columns)

# Combine all into one DataFrame
importance_df = pd.DataFrame({
    'Pearson R2': pearson_r2,
    'Std Coef': std_coefs.abs(),
    'Usefulness': usefulness,
    'SHAP (Linear)': shap_importance,
    'Johnson Epsilon': johnson,
    'RF Gini': rf_importance
})

# Normalize to 100% within each column
importance_percent = importance_df.divide(importance_df.sum()) * 100
importance_percent = importance_percent.round(1)

pd.set_option('display.max_rows', None)
display(importance_percent)

```


	Pearson R2	Std Coef	Usefulness	SHAP (Linear)	Johnson Epsilon	RF Gini
trust	15.6	25.3	31.5	26.7	32.8	15.6
build	8.8	4.4	1.0	4.5	1.0	10.2
differs	8.1	6.1	2.1	5.6	1.9	9.0
easy	10.8	4.8	1.1	5.1	1.2	10.0
appealing	10.3	7.4	2.7	7.6	2.8	8.6
rewarding	9.0	1.1	0.1	1.1	0.1	10.1
popular	7.0	3.6	0.8	3.8	0.7	9.5
service	15.0	19.3	17.9	19.9	19.1	13.0
impact	15.4	28.0	42.8	25.5	40.4	14.1

KNN: Visualizing Model Flexibility

To explore how the choice of model affects classification behavior, we implemented and visualized three models trained on yogurt brand choice data:

- A linear classifier (Logistic Regression)
- K-Nearest Neighbors with $K = 15$
- K-Nearest Neighbors with $K = 1$

The figure below replicates the visual style of Slide 19 from class, showing how each model divides the input space (p1 vs. p2) into regions corresponding to brand predictions.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from matplotlib.colors import ListedColormap

# Load yogurt dataset again and prepare
yogurt_data = yogurt_df.copy()
X = yogurt_data[['p1', 'p2']] # use two features for 2D visualization
y = yogurt_data[['y1', 'y2', 'y3', 'y4']].idxmax(axis=1) # assign class based on one-hot matrix

# Encode 'y1' -> 0, 'y2' -> 1, etc.
y = y.str.extract('(\d)').astype(int) - 1
y = y[0]
```

```

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# Train models
logreg = LogisticRegression().fit(X_train, y_train)
knn_15 = KNeighborsClassifier(n_neighbors=15).fit(X_train, y_train)
knn_1 = KNeighborsClassifier(n_neighbors=1).fit(X_train, y_train)

# Meshgrid for plotting decision boundaries
h = 0.01
x_min, x_max = X_train.iloc[:, 0].min() - 0.05, X_train.iloc[:, 0].max() + 0.05
y_min, y_max = X_train.iloc[:, 1].min() - 0.05, X_train.iloc[:, 1].max() + 0.05
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
grid = np.c_[xx.ravel(), yy.ravel()]

# Predict for each model
Z_logreg = logreg.predict(grid).reshape(xx.shape)
Z_knn15 = knn_15.predict(grid).reshape(xx.shape)
Z_knn1 = knn_1.predict(grid).reshape(xx.shape)

# Plotting
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
cm = ListedColormap(['#FFA07A', '#87CEFA', '#90EE90', '#DDA0DD'])

titles = ['Linear Regression', 'K=15', 'K=1']
models = [(Z_logreg, logreg), (Z_knn15, knn_15), (Z_knn1, knn_1)]

for ax, (Z, model), title in zip(axes, models, titles):
    ax.contourf(xx, yy, Z, cmap=cm, alpha=0.4)
    scatter = ax.scatter(X_train.iloc[:, 0], X_train.iloc[:, 1], c=y_train, cmap=cm, edgecolor='k')
    ax.set_title(title)
    ax.set_xlabel('p1')
    ax.set_ylabel('p2')

fig.suptitle("K NEAREST NEIGHBORS: Decision Boundaries (Slide 19 Style)", fontsize=16)
plt.tight_layout()
plt.show()

```

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/util

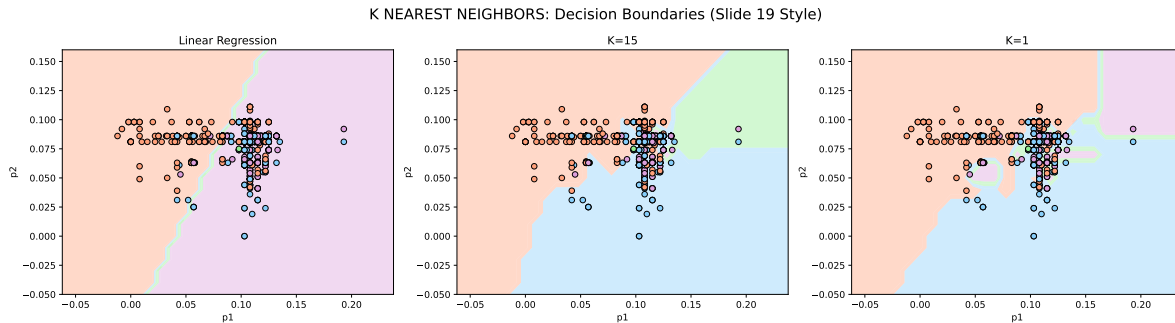
X does not have valid feature names, but LogisticRegression was fitted with feature names

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/util

X does not have valid feature names, but KNeighborsClassifier was fitted with feature names

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/util

X does not have valid feature names, but KNeighborsClassifier was fitted with feature names



We observe that the linear model is too constrained and fails to capture non-linear boundaries. KNN with $K=15$ provides a good balance between smoothness and adaptability. In contrast, $K=1$ overfits the training data, creating overly complex boundaries.

This highlights the classic **bias-variance trade-off** in machine learning.